Final project

December 10, 2025

1 DSC 200 Final project

[ ]:

```

```

This project involves the analysis of a climate projection of temperature change across the Earth under a 'middle-of-the-road' future scenario in which global mean temperatures reach more than 2 degrees centigrade above the pre-industrial. You will read in the data, analyze it, and visualize it in a variety of ways. You will also write a small command line interface to make the analysis more interactive.

We will be using data created by the NorESM2 climate model and processed and as part of the ClimateBench dataset, described in this paper. **All the files you will need for this project are available in the `public` folder on DataHub though.**

1.0.1 Table of contents:

1. Read in CO2 emissions data for historical + future scenario [5 points]
2. Read in temperature data [5 points]
3. Create a simple regression model [15 points]
4. Extend this to a regional temperature model, by region, and by state [15 points]
5. Plot the regression coefficients for each country [5 points]
6. Do an analysis of your choosing [15 points]
7. Make a command line interface [20 points]

1.0.2 Other requirements:

You will also be graded on Documentation and commenting, coding style, and code quality: - Documentation should be in the form of a Jupyter notebook, and should include a description of the data, the analysis, and the results. [10 points] - The code should be well documented, and should follow the PEP8 coding style. [5 points] - The code should be well organized, and should be broken up into functions and classes as appropriate. For full marks try to use no for-loops in your code. [5 points]

Be sure to read the question and reach out to the instructor or TA if you have any questions.

1.0.3 Total points: 100 (30% of total), plus midterm makeup

- Note, the midterm grade is still capped at 100%

1.0.4 Deadline: Wednesday December 10th 11:59pm

[1]:
```
import numpy as np
import pandas as pd
import xarray as xr
import matplotlib.pyplot as plt
```

1.1 1. Read in historical and future (estimated) CO2 emissions data

Monthly CO2 emissions data is available since 1850 globally for each industrial sector. We want to use annual average totals of the emissions for all sectors for our analysis. Since CO2 has a very long lifetime in the atmosphere (1000's of years), we can assume that the total amount of anthropogenic CO2 in the atmosphere is the cumulative sum of all emissions since 1850. This is what we will use for our analysis.

To read this data do **either** Q1a (to get 5 points plus additional makeup points for the midterm) **or** Q1b (to get 5 points for this project)

[2]:
```
input_path = 'public/'
```

1.1.1 1a. OPTIONAL: Create interpolated cumulative CO2 from the raw data using Pandas

To gain (up to) 20% additional marks for your midterm makeup (capped at 100%), you can create a new column in the CO2 emissions data that is the cumulative CO2 emissions.

[3]:
```
# These input files provide CO2 emissions data for the historical period␣
↪(1850-2014) and the future period (2015-2100). They should all be␣
↪concatenated into a single file.

historical_input_files = ['CO2_emissions/
↪CO2-em-anthro_input4MIPs_emissions_CMIP_CEDS-2017-05-18_gn_200001-201412.
↪csv',
                          'CO2_emissions/
↪CO2-em-anthro_input4MIPs_emissions_CMIP_CEDS-2017-05-18_gn_195001-199912.
↪csv',
                          'CO2_emissions/
↪CO2-em-anthro_input4MIPs_emissions_CMIP_CEDS-2017-05-18_gn_190001-194912.
↪csv',
                          'CO2_emissions/
↪CO2-em-anthro_input4MIPs_emissions_CMIP_CEDS-2017-05-18_gn_185101-189912.
↪csv']

future_input_file = 'CO2_emissions/
↪CO2-em-anthro_input4MIPs_emissions_ScenarioMIP_IAMC-MESSAGE-GLOBIOM-ssp245-1-1_gn_201501-21
↪csv'
```

First you will need to read, concatenate and process the raw CSV files, and sum over the `sector`

and month columns to get an annual total.

[4]:
```
# YOUR CODE HERE

# load and concatenate the data
df_hist = pd.concat([pd.read_csv(input_path + f) for f in␣
 ↪historical_input_files], ignore_index=True)

# load future data
df_future = pd.read_csv(input_path + future_input_file)

# combine historical and future data
df_complete = pd.concat([df_hist, df_future], ignore_index=True)

# convert the complete DataFrame to an Xarray Dataset for 2+ dimensions of data
ds = df_complete.set_index(["year", "month", "sector"]).to_xarray()

ds

# raise NotImplementedError()
```

[4]:
```
<xarray.Dataset> Size: 453kB
Dimensions:       (year: 174, month: 12, sector: 9)
Coordinates:
  * year          (year) int64 1kB 1851 1852 1853 1854 … 2070 2080 2090 2100
  * month         (month) int64 96B 1 2 3 4 5 6 7 8 9 10 11 12
  * sector        (sector) object 72B 'AGR' 'ENE' 'IND' … 'SLV' 'TRA' 'WST'
Data variables:
    em            (year, month, sector) object 150kB 'CO2' 'CO2' … 'CO2' 'CO2'
    global_total  (year, month, sector) float64 150kB 0.0 … 1.108e+04
    units         (year, month, sector) object 150kB 'kt' 'kt' … 'kt' 'kt'
```

[5]:
```
# sum over the sector and month columns to get an annual total
annual = ds["global_total"].sum(dim=("month", "sector"))

# add to existing dataset
ds = ds.assign(annual_emissions=annual)

ds
```

[5]:
```
<xarray.Dataset> Size: 454kB
Dimensions:           (year: 174, month: 12, sector: 9)
Coordinates:
  * year              (year) int64 1kB 1851 1852 1853 1854 … 2080 2090 2100
  * month             (month) int64 96B 1 2 3 4 5 6 7 8 9 10 11 12
  * sector            (sector) object 72B 'AGR' 'ENE' 'IND' … 'TRA' 'WST'
Data variables:
```

```
    em                (year, month, sector) object 150kB 'CO2' 'CO2' … 'CO2'
    global_total      (year, month, sector) float64 150kB 0.0 … 1.108e+04
    units             (year, month, sector) object 150kB 'kt' 'kt' … 'kt' 'kt'
    annual_emissions  (year) float64 1kB 1.889e+05 1.966e+05 … 1.258e+07
```

Note, the future data is only provided every five years so that will need linearly interpolated to get annual values.

[6]:
```
# YOUR CODE HERE

#expand the dataset to hold data expansion
ds = ds.reindex(year=np.arange(1851, 2101))

# linearly interpolate the gappy annual data to fill in missing years␣
↪(1851-2100)
ds_interpolated = annual.interp(year=np.arange(1851, 2101))

# calculate the cumulative co2 emissions from the continuous annual data
cumulative_co2_interpolated = ds_interpolated.cumsum(dim="year")

# add to dataset
ds = ds.assign(cumulative_co2=cumulative_co2_interpolated)

# raise NotImplementedError()
```

Now, divide by 1e6 to get the units in GtC (Giga tonnes of carbon) and calculate the cumulative sum.

[7]:
```
# YOUR CODE HERE

# convert to giga tonnes
annual_gtc = ds_interpolated / 1e6

# calculate the cumulative sum
annual_gtc_cumsum = annual_gtc.cumsum(dim="year")

# add to dataset
ds = ds.assign(cumulative_co2_GtC=annual_gtc_cumsum)

ds

# raise NotImplementedError()
```

[7]:
```
<xarray.Dataset> Size: 656kB
Dimensions:             (year: 250, month: 12, sector: 9)
Coordinates:
  * year                (year) int64 2kB 1851 1852 1853 1854 … 2098 2099 2100
  * month               (month) int64 96B 1 2 3 4 5 6 7 8 9 10 11 12
```

```
  * sector                (sector) object 72B 'AGR' 'ENE' 'IND' … 'TRA' 'WST'
Data variables:
    em                    (year, month, sector) object 216kB 'CO2' 'CO2' … 'CO2'
    global_total          (year, month, sector) float64 216kB 0.0 … 1.108e+04
    units                 (year, month, sector) object 216kB 'kt' 'kt' … 'kt'
    annual_emissions      (year) float64 2kB 1.889e+05 1.966e+05 … nan 1.258e+07
    cumulative_co2        (year) float64 2kB 1.889e+05 3.855e+05 … 4.356e+09
    cumulative_co2_GtC    (year) float64 2kB 0.1889 0.3855 … 4.344e+03 4.356e+03
```

Check the data against the existing `combined_co2.csv` and save it to use for the rest of the project.

[8]:
```python
# YOUR CODE HERE

# convert to dataframe
df_calculated = ds['cumulative_co2_GtC'].to_dataframe().reset_index()

# load existing file to dataframe
df_existing = pd.read_csv("combined_co2.csv")

# merge the two DataFrames for comparison
df_comparison = df_calculated.merge(df_existing, on='year', how='outer').
 ↪dropna()

# comparing floats adjust for float error
float_error = 1e-6

# create a new comparison column
df_comparison['difference'] = df_comparison['cumulative_co2_GtC'] -␣
 ↪df_comparison['cumulative_CO2']

# filter values with difference
values_different = values_different = df_comparison[df_comparison['difference']␣
 ↪> float_error]

# if no difference save
if len(values_different) == 0:
    df_calculated = df_calculated.rename(columns={
        'cumulative_co2_GtC': 'cumulative_CO2'
    }
)
    df_calculated.to_csv("combined_co2.csv", index=False)
    print("no errors found, adding new data to combined_co2.csv")
else:
    print(values_different)

# raise NotImplementedError()
```

```
no errors found, adding new data to combined_co2.csv
```

1.1.2 1b. Otherwise just read in the cumulative CO2 emissions data from the provided file

[9]:
```
pre_processed_input_file = input_path+'cumulative_co2.csv'

# Read the input files
```

[10]:
```
# YOUR CODE HERE
# raise NotImplementedError()
```

1.2 2. Read in the temperature data

Note, this temperature change as modelled by the NorESM2 climate model relative to the pre-industrial period. It's purely driven by the prescribed emissions, so it won't perfect represent the actual temperatures we experienced in a given year (which are subject to chaotic fluctuations), but it's a good model.

[11]:
```
temperature_input_file = input_path+'global_temperature.nc'

# Read the input files
## Note, the variable name in the netcdf file is 'tas' (not 'temperature')
```

[12]:
```
# YOUR CODE HERE
# load into xarray dataset
ds_temperature = xr.open_dataset(temperature_input_file)

# rename the temperature 'tas' to 'temperature' for clarity
ds_temperature = ds_temperature.rename({'tas': 'temperature'})

ds_temperature
# raise NotImplementedError()
```

[12]:
```
<xarray.Dataset> Size: 14MB
Dimensions:      (time: 250, lat: 96, lon: 144)
Coordinates:
  * time         (time) int64 2kB 1851 1852 1853 1854 … 2097 2098 2099 2100
  * lat          (lat) float64 768B -90.0 -88.11 -86.21 … 86.21 88.11 90.0
  * lon          (lon) float64 1kB 0.0 2.5 5.0 7.5 … 350.0 352.5 355.0 357.5
    quantile     float64 8B …
Data variables:
    temperature  (time, lat, lon) float32 14MB …
```

And take the global mean. Don't forget to calculate and include weights for the latitude of each grid cell.

[13]:
```
# YOUR CODE HERE

# calculate the latitude weights (radians)
weights = np.cos(np.deg2rad(ds_temperature['lat']))

# apply weights
temp_weighted = ds_temperature['temperature'].weighted(weights)

# calculate weighted mean
ds_global_temp = temp_weighted.mean(dim=['lat', 'lon'])

# add back to dataset
ds_temperature = ds_temperature.assign(global_mean=ds_global_temp)

ds_temperature
# raise NotImplementedError()
```

[13]:
```
<xarray.Dataset> Size: 14MB
Dimensions:      (time: 250, lat: 96, lon: 144)
Coordinates:
  * time         (time) int64 2kB 1851 1852 1853 1854 … 2097 2098 2099 2100
  * lat          (lat) float64 768B -90.0 -88.11 -86.21 … 86.21 88.11 90.0
  * lon          (lon) float64 1kB 0.0 2.5 5.0 7.5 … 350.0 352.5 355.0 357.5
    quantile     float64 8B …
Data variables:
    temperature  (time, lat, lon) float32 14MB -0.6673 -0.667 … 5.264 5.266
    global_mean  (time) float64 2kB 0.06726 0.12 0.1928 … 2.187 2.148 2.166
```

1.3 3. Create a simple regression model of global warming

Global warming can be surprisingly well predicted just using a linear model of cumulative CO2 emissions. This is because the CO2 emissions are the primary driver of global warming, and the CO2 stays in the atmosphere for a long time (see e.g. https://www.nature.com/articles/ngeo3031).

To get global temperature as a function of cumulative CO2. You can use the `LinearRegression` class from `sklearn.linear_model`, with documentation provided here. You should only need to use the `fit` and `predict` methods. The `fit` method takes two arguments, the first is the input data, and the second is the output data. The `predict` method takes one argument, the input data.

Alternatively, you can also use the `statsmodels` package to get more detailed statistics on the regression. See here for documentation.

Since we're only aiming to create an interpolation model, we don't need to worry too much about keeping a test set aside. We can just use all the data to train the model. You could also use a train-test split if you want to.

[14]:
```
import json
import os
```

```python
def update_coefficients_json_by_key(key_id,slope,intercept,r_squared,filename):
    """
    Reads an existing JSON file, merges a single new set of coefficient data
    under a specified key (country/global ID), and overwrites the file.

    Args:
        key_id: The unique identifier for the entry .
        slope: The calculated slope coefficient.
        intercept: The calculated intercept coefficient.
        r_squared: The calculated R-squared value.
        filename: The name of the JSON file to update.
    """

    # structure the new entry
    new_entry = {
        key_id: {
            "slope": slope,
            "intercept": intercept,
            "r_squared": r_squared
        }
    }

    # intialize the existing data
    existing_data = {}

    # load existing data if possible
    if os.path.exists(filename):
        try:
            with open(filename, 'r') as f:
                existing_data = json.load(f)
        except json.JSONDecodeError:
            print(f"Warning: File '{filename}' exists but is empty or invalid␣
↪JSON. Starting fresh.")

    # add or update the existing data with the new entry
    existing_data.update(new_entry)

    # save file
    try:
        with open(filename, 'w') as f:
            json.dump(existing_data, f, indent=4)

        print(f"Data for '{key_id}' successfully merged and saved to:␣
↪'{filename}'")

    except Exception as e:
        print(f"Error saving JSON file '{filename}': {e}")
```

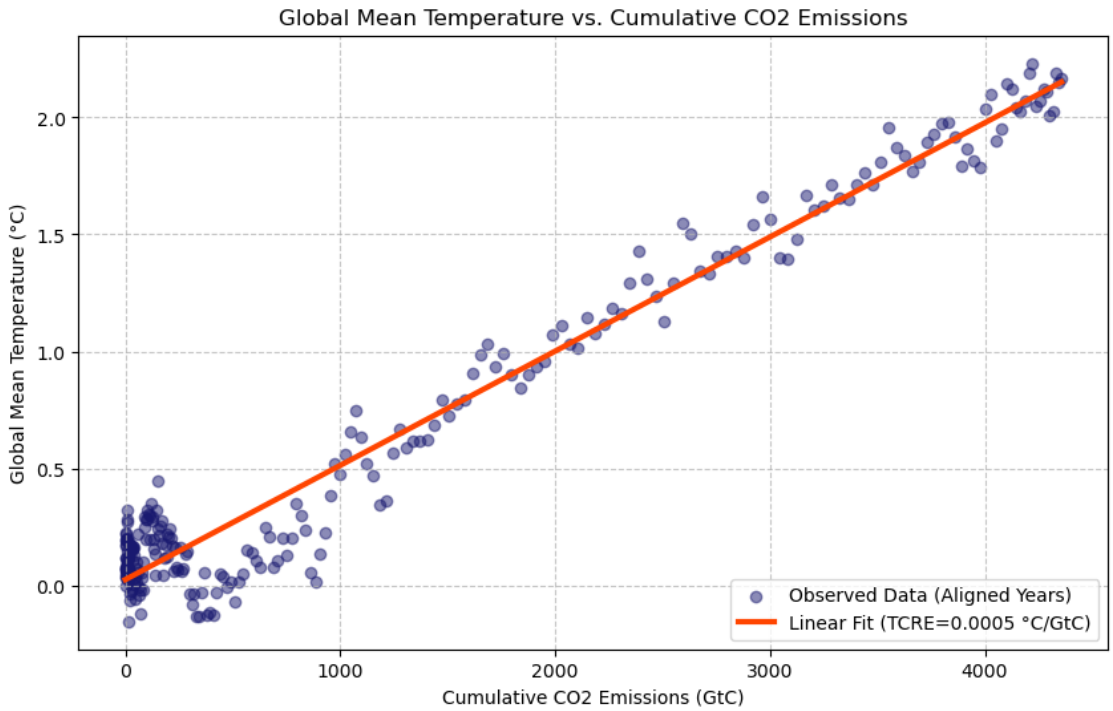
[15]:
```
# YOUR CODE HERE
import json
import numpy as np
from sklearn.linear_model import LinearRegression

# align the 2 data sets
X_data_aligned, Y_data_aligned = xr.align(
    ds['cumulative_co2_GtC'],
    ds_temperature['global_mean'],
    join='inner', # Only keep years present in both datasets
    fill_value=np.nan # Use NaN for missing data (though inner join prevents␣
 ↪this)
)

# convert to 2d numpy array
X = X_data_aligned.values.reshape(-1, 1)

# assign y
y = Y_data_aligned.values

# create + train
model = LinearRegression()
model.fit(X, y)

#extract + save the intercepts(Part 7)
save_file_name = "trained_linear_coefficients.json"

# extract coefficients from the fitted model
slope = model.coef_[0]
intercept = model.intercept_
r_squared = model.score(X, y)

update_coefficients_json_by_key("Global",slope,intercept,r_squared,save_file_name)
```

```
Data for 'Global' successfully merged and saved to:
'trained_linear_coefficients.json'
```

Plot global mean temperature as a function of cumulative CO2 emissions, along with the regression fit

[16]:
```
# YOUR CODE HERE

# regression fit values
Y_pred = model.predict(X)

# initialize plot
plt.figure(figsize=(10, 6))
```

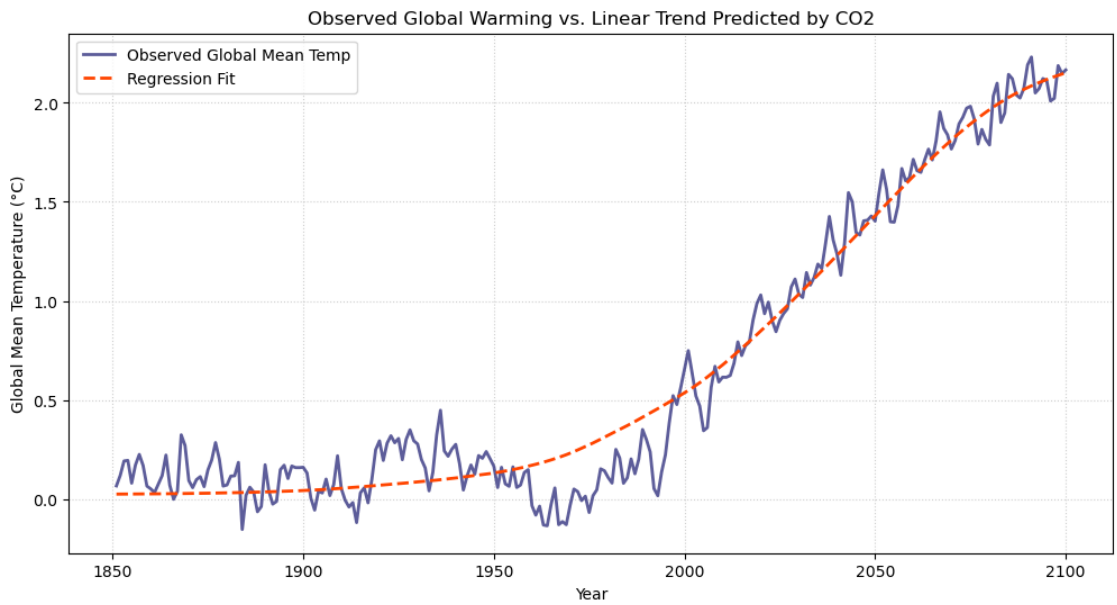
```python
# create scatter plot
plt.scatter(
    X.flatten(),
    y,
    label='Observed Data (Aligned Years)',
    color='midnightblue',
    alpha=0.5
)

# add prediciton line
plt.plot(
    X.flatten(),
    Y_pred,
    color='orangered',
    linewidth=3,
    label=f'Linear Fit (TCRE={model.coef_[0]:.4f} °C/GtC)'
)

# label axiis
plt.xlabel('Cumulative CO2 Emissions (GtC)')
plt.ylabel('Global Mean Temperature (°C)')
plt.title('Global Mean Temperature vs. Cumulative CO2 Emissions')
plt.grid(True, linestyle='--', alpha=0.7)
plt.legend(loc='lower right')

# show plot
plt.show()
# raise NotImplementedError()
```



Plot global mean temperature as a function of year, along with the regression fit

[17]:
```
# YOUR CODE HERE

# extract years
years = Y_data_aligned['time'].values

# extract temps
Y_observed = Y_data_aligned.values

# intialize plot
plt.figure(figsize=(12, 6))

# plot observed temp
plt.plot(years, Y_observed, label='Observed Global Mean Temp',␣
 ↪color='midnightblue', alpha=0.7, linewidth=2)

# plot prediction/regression fit
plt.plot(years, Y_pred, color='orangered', linestyle='--', linewidth=2,
         label='Regression Fit')

# title + labels
plt.xlabel('Year')
plt.ylabel('Global Mean Temperature (°C)')
```

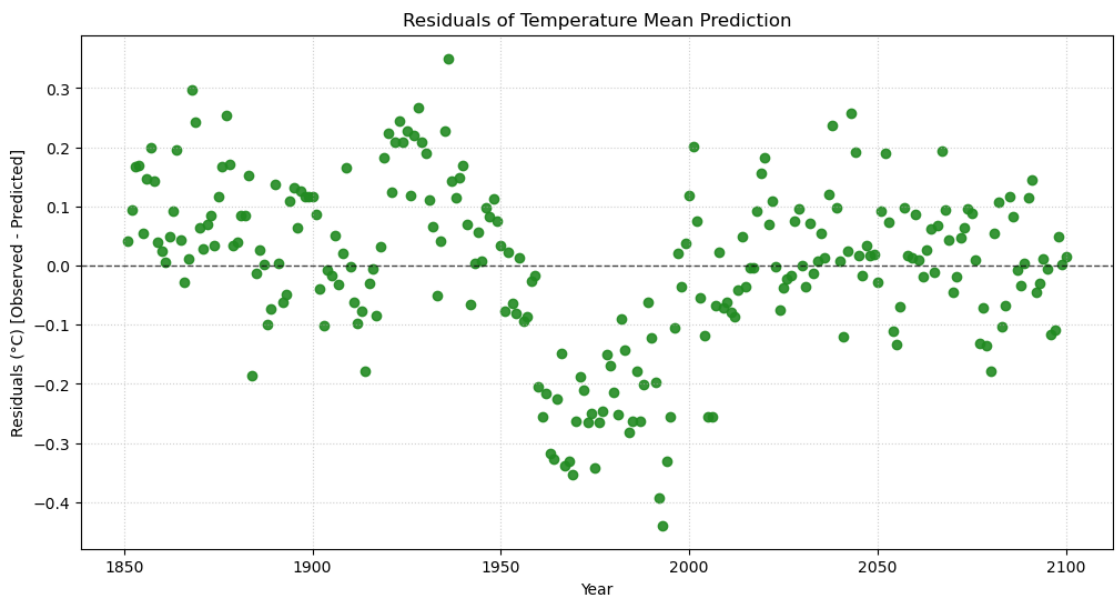
```
plt.title('Observed Global Warming vs. Linear Trend Predicted by CO2')
plt.grid(True, linestyle=':', alpha=0.6)
plt.legend(loc='upper left')

# show plot
plt.show()
# raise NotImplementedError()
```



Where does the prediction do well? Where does it do less well? Visualise the residuals.

[18]:
```
# YOUR CODE HERE

# ----------where does it do well-----------------
# => predicts long term trends for 25-50 years

# ----------where it doesnt do well--------------
# => short term yearly predictions
# => over predicited global warming during the 1950s - 1980

# calculate residuals
residuals = Y_observed - Y_pred

# initialize plot
plt.figure(figsize=(12, 6))

# plot residuals vs. Year
# basis line (0 error).
```

```
plt.axhline(0, color='black', linestyle='--', linewidth=1, alpha=0.7)

# plot residuals
plt.scatter(years, residuals, color='forestgreen', alpha=0.9)

# labels + titles
plt.xlabel('Year')
plt.ylabel('Residuals (°C) [Observed - Predicted]')
plt.title('Residuals of Temperature Mean Prediction')
plt.grid(True, linestyle=':', alpha=0.6)

# show plot
plt.show()

# raise NotImplementedError()
```



1.4 4. Extend this to a regional temperature model, by region, and by state

While the relationship between global temperature and cumulative CO2 emissions is very linear, the relationship between regional temperature and cumulative CO2 emissions is less so. This is because the regional temperature is affected by other factors, such as the regional distribution of land and ocean, and the regional distribution of CO2 emissions. Nevertheless, let's see how well it can do

Read in the country mask which is a 2D array of the same size as the temperature data, with each grid cell containing the country code of the country that grid cell is in.

[19]:
```
country_mask_file = input_path+'country_mask.nc'
```

Average the spatial coordinates into countries so that you end up with a dataset that has dimensionality of the number of countries by the number of time points.

[20]:
```
# YOUR CODE HERE

# import data into xarray
ds_mask = xr.open_dataset(country_mask_file)

ds_mask

#rename the abritray variable name for easier reference
ds_mask = ds_mask.rename({'__xarray_dataarray_variable__': 'country_code'})

ds_mask

# raise NotImplementedError()
```

[20]:
```
<xarray.Dataset> Size: 1MB
Dimensions:       (lat: 96, lon: 144)
Coordinates:
  * lat           (lat) float64 768B -90.0 -88.11 -86.21 … 86.21 88.11 90.0
  * lon           (lon) float64 1kB 0.0 2.5 5.0 7.5 … 350.0 352.5 355.0 357.5
Data variables:
    country_code  (lat, lon) <U24 1MB …
```

[21]:
```
# YOUR CODE HERE
# extract data array by grouping
country_mask_da = ds_mask['country_code']

# calculate mean based off country code, since multi-dimensional use stacked
ds_country_means = (
    ds_temperature['temperature']
    .groupby(country_mask_da)
    .mean(dim='stacked_lat_lon')
)

# convert the resulting DataArray back to a Dataset
ds_country_means = ds_country_means.to_dataset(name='country_mean_temp')

# rename country code to countries for easy ref
ds_country_means = ds_country_means.rename({'country_code': 'country'})

#rename time to year for dataset snyc for co2 emissions
ds_country_means = ds_country_means.rename({'time': 'year'})

ds_co2 = ds[['cumulative_co2_GtC']]
```

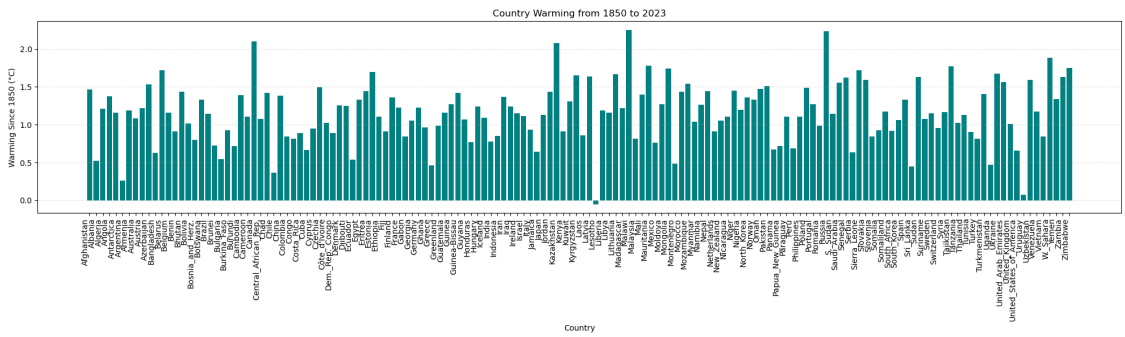
```
ds_country_means = ds_country_means.merge(ds_co2)

ds_country_means
# raise NotImplementedError()
```

[21]:
```
<xarray.Dataset> Size: 155kB
Dimensions:              (year: 250, country: 150)
Coordinates:
  * year                 (year) int64 2kB 1851 1852 1853 1854 … 2098 2099 2100
    quantile             float64 8B …
  * country              (country) object 1kB 'Afghanistan' … 'Zimbabwe'
Data variables:
    country_mean_temp    (year, country) float32 150kB 0.3524 0.2269 … 3.674
    cumulative_co2_GtC   (year) float64 2kB 0.1889 0.3855 … 4.344e+03 4.356e+03
```

Plot a bar chart of the warming in each country in 2023. Note, the temperature data is baselined to 1850.

[22]:
```
# YOUR CODE HERE

# get baseline mean temp
temp_base = ds_country_means.sel(year=1851)['country_mean_temp']

# get 2023 temp
temp_current = ds_country_means.sel(year=2023)['country_mean_temp']

#calculate warming
warming_2023 = temp_current - temp_base

# convert to df
df_warming = warming_2023.to_dataframe(name='warming_degC').reset_index()

# intialize plot
plt.figure(figsize=(20, 6))

# plot
plt.bar(
    df_warming['country'],
    df_warming['warming_degC'],
    color='teal'
)

# labeling and formatting
plt.xlabel('Country')
plt.ylabel('Warming Since 1850 (°C)')
plt.title('Country Warming from 1850 to 2023')
plt.xticks(rotation=90, ha='right')
```

```
plt.grid(axis='y', linestyle=':', alpha=0.6)
plt.tight_layout()

# show plot
plt.show()
# raise NotImplementedError()
```



Calculate a linear regression model for each country along with the R^2 value. Plot the R^2 values for each country as a bar chart.

[23]:
```
# YOUR CODE HERE
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

# define regression for lambda in df
def linregress_country(data):
    """
    Performs linear regression on Temperature vs. Time for a single country.
    Returns the R-squared value and the slope (warming rate).
    """

    # drop NaN
    required_cols = ['cumulative_co2_GtC', 'country_mean_temp']
    data = data.dropna(subset=required_cols)

    # define year as x
    x = data['cumulative_co2_GtC'].values
    # defin temp as y
    y = data['country_mean_temp'].values

    # reshape for sklearn
    X = x.reshape(-1, 1)

    # initialize and fit the model
    model = LinearRegression()
```

```
    model.fit(X, y)

    # extract coefficients + r^2
    slope = model.coef_[0]
    intercept = model.intercept_
    r_squared = model.score(X, y)

    # return r^2 + slope + intercepts
    return pd.Series({
        'r_squared': r_squared,
        'slope': slope,
        'intercept': intercept
    })
```

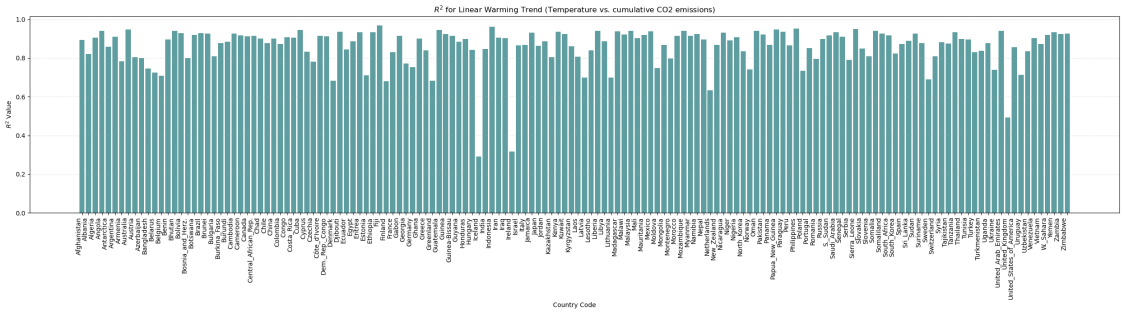
[24]:
```
# YOUR CODE HERE

# save trained linear coefficents (part 7)

def save_coefficients_to_json(df_results, filename):
    """
    Reads the existing coefficient JSON file (preserving entries),
    merges all country-level coefficients from the DataFrame into it, and saves
    the combined structure.

    Args:
        df_results: DataFrame containing 'country', 'slope',
                                    'intercept', and 'r_squared' for countries.
        filename: The name of the file to save the JSON data to.
    """

    # initialize storage for all
    existing_data = {}

    # update storage with entries
    if os.path.exists(filename):
        try:
            with open(filename, 'r') as f:
                existing_data = json.load(f)
        except json.JSONDecodeError:
            print(f"Warning: File '{filename}' exists but is invalid JSON.␣
↪Starting country list fresh.")

    # set the country as the key index
    df_indexed = df_results.set_index('country', drop=True).copy()

    # convert df to dictionary
    country_data_to_save = df_indexed.to_dict(orient='index')
```

```
    # add or update storage with new data
    existing_data.update(country_data_to_save)
    data_to_save = existing_data

    try:
        with open(filename, 'w') as f:
            json.dump(data_to_save, f, indent=4)

        print(f"Total entries saved/updated: {len(data_to_save)}")

    except Exception as e:
        print(f"Error saving JSON file: {e}")
```

[25]:
```
# YOUR CODE HERE

# flatten dataset into pandas dataframe
df_regress = ds_country_means.to_dataframe().reset_index()

# group by country and apply the regression function
df_results = df_regress.groupby('country')[['cumulative_co2_GtC',␣
 ↪'country_mean_temp']].apply(linregress_country).reset_index()

# save trained linear coefficents (part 7)
save_file_name = "trained_linear_coefficients.json"
save_coefficients_to_json(df_results,save_file_name)

# initialize plot
plt.figure(figsize=(25, 7))

# plot bars
plt.bar(df_results['country'], df_results['r_squared'], color='cadetblue')

# label + formatting
plt.xlabel('Country Code')
plt.ylabel(r'$R^{2}$ Value')
plt.title(r'$R^{2}$ for Linear Warming Trend (Temperature vs. cumulative CO2␣
 ↪emissions)')
plt.xticks(rotation=90, ha='right')
plt.grid(axis='y', linestyle=':', alpha=0.6)
plt.tight_layout()

# show plot
plt.show()

# not sure why there is a quesiton on the bottom but no answer box:
```

```
# sort the ascending by error
df_results = df_results.sort_values(by='r_squared', ascending=False)

# select 3 from the top
top_least_error = df_results.head(3)
print("Linear regression works really well with:")
print(top_least_error)

# select 3 from the bottom
worst_error = df_results.tail(3)
print("Linear regression works not that well with:")
print(worst_error)

# seems like trend for being good for southern hemisphere countries but not␣
↪northern

# raise NotImplementedError()
```

```
Total entries saved/updated: 152
```




```
Linear regression works really well with:
         country  r_squared     slope  intercept
45          Fiji   0.968093  0.000490   0.108601
62     Indonesia   0.961263  0.000489   0.063197
108  Philippines   0.951514  0.000502   0.052144
Linear regression works not that well with:
            country  r_squared     slope  intercept
140  United_Kingdom   0.493429  0.000213  -0.024861
65           Ireland   0.317804  0.000147   0.011695
60           Iceland   0.291359  0.000380   0.120460
```

For which countries does the linear assumption work well, and where does it work less well?

YOUR ANSWER HERE

1.5 5. Plot the regression coefficients for each country

Which five countries are most sensitive to CO2 emissions and hence warming the fastest?

[26]:
```
# YOUR CODE HERE

#sort the warming data ascending by warming degress
df_warming = df_results.sort_values(by='slope', ascending=False)

#select 5 from the top
top_5_warming = df_warming.head(5)

print("Top 5 Countries most effected by CO2 Emissions:")
print(top_5_warming)

# raise NotImplementedError()
```

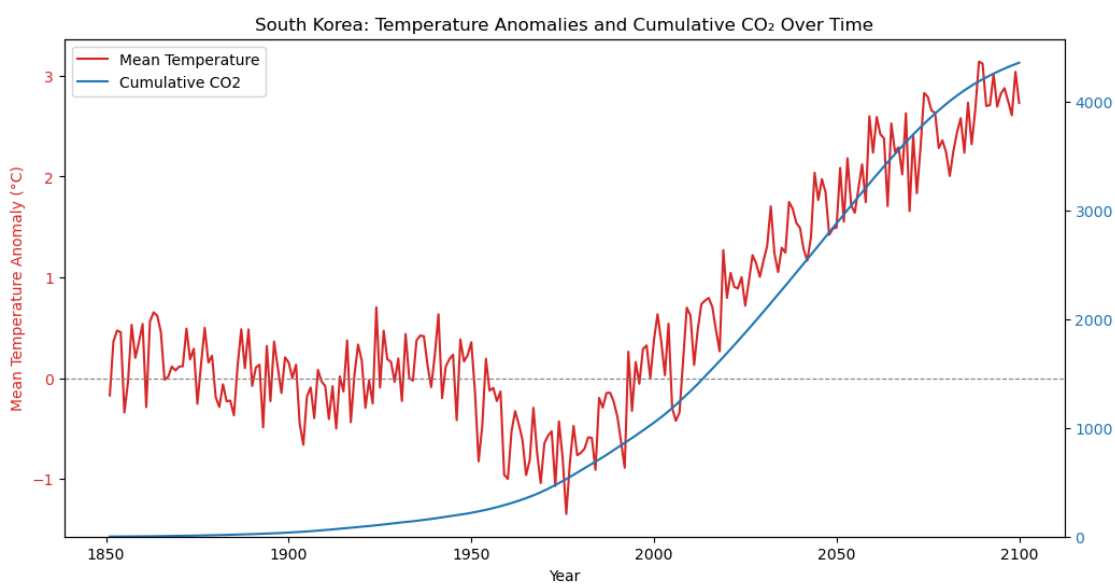
```
Top 5 Countries most effected by CO2 Emissions:
                  country  r_squared     slope  intercept
25                 Canada   0.911913  0.001087   0.113428
112                Russia   0.898797  0.000988   0.230603
114          Saudi_Arabia   0.932939  0.000932  -0.054870
139  United_Arab_Emirates   0.941618  0.000912  -0.026674
73                 Kuwait   0.924279  0.000879  -0.076896
```

1.6 6. Do an analysis of your choosing

Maybe dig into the changes in one particular country, or look at changes in the variability of temperature. Perhaps look at the chances of exceeding certain temperature limits.

[27]:
```
# YOUR CODE HERE

# Analysis: South Korea Temperature & CO2

# select South Korea data from the country-mean dataset
df_korea = ds_country_means.sel(country = "South_Korea")

# initialize plot
fig, ax1 = plt.subplots(figsize=(12, 6))
ax1.set_xlabel('Year')

# plot temperature
color_temp = 'tab:red'
ax1.plot(df_korea['year'], df_korea['country_mean_temp'], color=color_temp,␣
 ↪label='Mean Temperature')
ax1.tick_params(axis='y', labelcolor=color_temp)

# add horizontal line at 0 for baseline reference
ax1.axhline(0, color='gray', linestyle='--', linewidth=0.8)
```

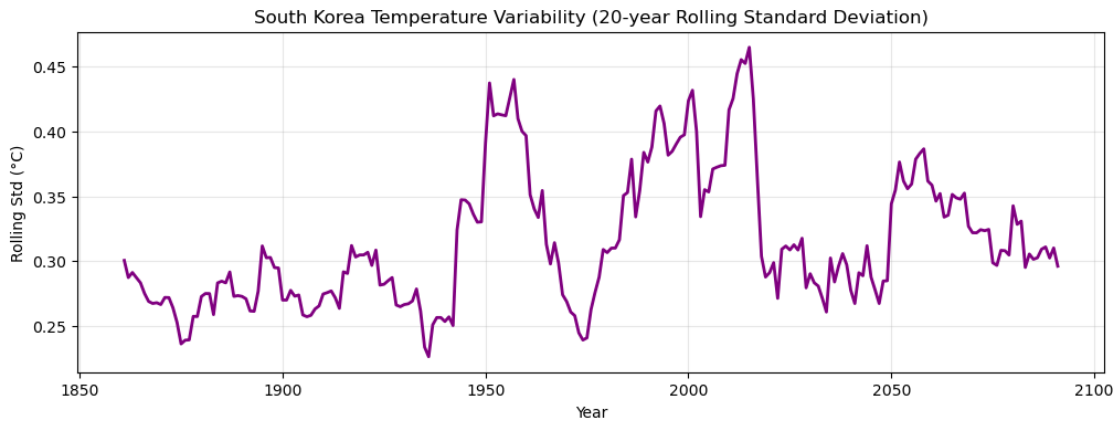
```python
# plot 2: Cumulative CO  emissions (2nd axis)
ax2 = ax1.twinx()
color_co2 = 'tab:blue'

ax2.plot(df_korea['year'], df_korea['cumulative_co2_GtC'], color=color_co2,␣
 ↪linestyle='-', label='Cumulative CO2')
ax2.tick_params(axis='y', labelcolor=color_co2)
ax2.set_ylim(bottom=0) # cumulative values should start at zero

# axis label for temperature
ax1.set_ylabel("Mean Temperature Anomaly (°C)", color=color_temp)

# figure title
plt.title("South Korea: Temperature Anomalies and Cumulative CO  Over Time")

# combine legends from both y-axes
lines = [l for l in (ax1.get_lines() + ax2.get_lines()) if not l.get_label().
 ↪startswith('_')]
labels = [l.get_label() for l in lines]
ax1.legend(lines, labels, loc='upper left')

plt.show()

#raise NotImplementedError()
```



[28]:
```
# south korea's long-term warming trend
# calculate the long term probability of exceeding high temperature thresholds
# see changes in temperature variability over time

# convert xarray fields into numpy arrays
years = df_korea['year'].values
temps = df_korea['country_mean_temp'].values

# remove any NaN values
mask = ~np.isnan(temps)
years_clean = years[mask]
temps_clean = temps[mask]

# (A) South Korea's long-term warming trend
# warming in °C per year, so multiply by 10 to express it per decade
coef = np.polyfit(years_clean, temps_clean, 1)
trend_per_year = coef[0]
trend_per_decade = trend_per_year * 10

print(f"Estimated warming trend in South Korea: {trend_per_decade:.4f} °C per␣
↪decade")

# (B) probability of exceeding a temperature threshold
# fraction of years in which South Korea's temperature anomaly exceeded a given␣
↪threshold
threshold = 1.5
prob_exceed = np.mean(temps_clean > threshold)

print(f"Probability that South Korea exceeded +{threshold} °C anomaly:␣
↪{prob_exceed:.3f}")

# (C) changes in temperature variability (20-year rolling std)
df_korea['rolling_std_20yr'] = (
    df_korea['country_mean_temp']
    .rolling(year=20, center=True)
    .std()
)

# plot the 20-year rolling variability
plt.figure(figsize=(12, 4))
plt.plot(df_korea['year'], df_korea['rolling_std_20yr'], color='purple',␣
↪linewidth=2)
plt.title('South Korea Temperature Variability (20-year Rolling Standard␣
↪Deviation)')
plt.xlabel('Year')
plt.ylabel('Rolling Std (°C)')
plt.grid(True, alpha=0.3)
```

```
plt.show()
```

```
Estimated warming trend in South Korea: 0.1102 °C per decade
Probability that South Korea exceeded +1.5 °C anomaly: 0.232
```



[29]:
```
# YOUR CODE HERE

# some interesting trends: korea went into a massive temperature downfall after␣
↪1950

# find the lowest mean temp
lowest_temp_value = df_korea['country_mean_temp'].min()

# find the corresponding year of that minimum temperature
year_of_lowest_temp = df_korea['country_mean_temp'].idxmin('year').item()

print(f"Year of Lowest Temperature: {year_of_lowest_temp}")
print(f"Absolute Lowest Temperature Value: {lowest_temp_value:.4f} °C")
```

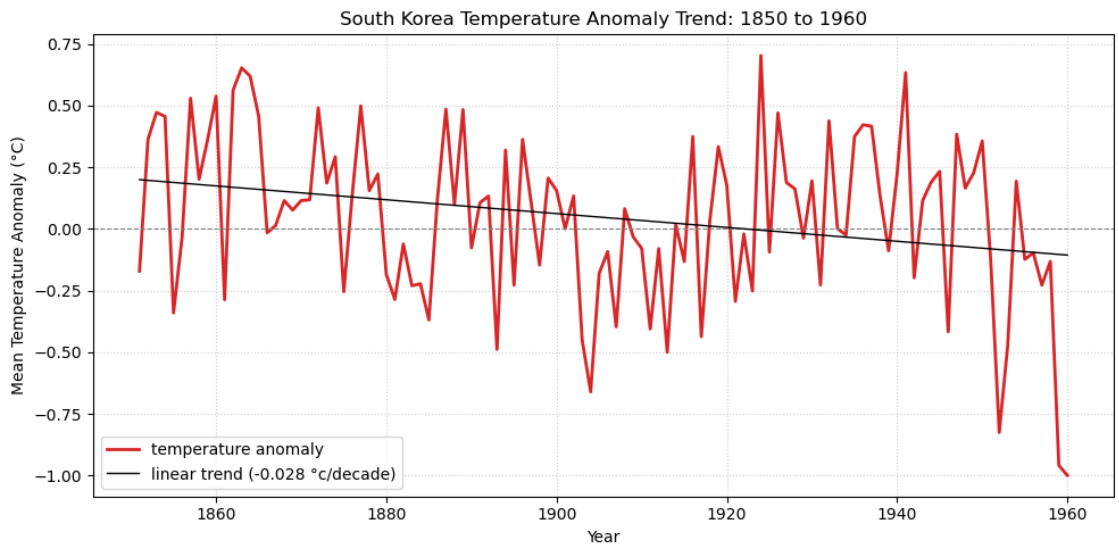
```
Year of Lowest Temperature: 1976.0
Absolute Lowest Temperature Value: -1.3480 °C
```

[30]:
```
# YOUR CODE HERE

# we choose an early historical period for south korea (1850-1960)
# this period roughly represents years before rapid industrial development
# we are to look at long-term background climate behavior without the strong␣
↪modern warming influence
start_year = 1850
end_year = 1960

# select the years in the chosen range from the xarray dataset
```

```
early_period = df_korea.sel(year=slice(start_year, end_year))

# convert the xarray object into a pandas dataframe
early_period_df = early_period.to_dataframe().reset_index()

# create a figure
fig, ax = plt.subplots(figsize=(10, 5))

# plot the temperature anomaly values year by year
ax.plot(
    early_period_df["year"],
    early_period_df["country_mean_temp"],
    color="tab:red",
    linewidth=2,
    label="temperature anomaly"
)

# draw a horizontal line at 0°c anomaly to help compare positive vs negative␣
↪anomaly years
# this makes it easier to see how far temperatures were from the baseline␣
↪climate
ax.axhline(0, color="gray", linestyle="--", linewidth=0.8)

# arrays for linear regression: X contains the years, and y contains the␣
↪temperature anomalies
X = early_period_df["year"].values
y = early_period_df["country_mean_temp"].values

# fit a first-degree polynomial (a straight line) to estimate the trend:
# the regression finds coefficients for: anomaly = a * year + b
trend_coeffs = np.polyfit(X, y, 1)
trend_line = np.poly1d(trend_coeffs)

# convert the slope from °c per year to °c per decade, because this is a more␣
↪interpretable rate
# positive slope means warming, negative means cooling.
slope_per_decade = trend_coeffs[0] * 10

print(f"estimated slope ({start_year}-{end_year}): {slope_per_decade:.4f} °c␣
↪per decade")

# plot the fitted trend line on the same graph
ax.plot(
    X,
    trend_line(X),
    color="black",
    linestyle="-",
```

```
    linewidth=1,
    label=f"linear trend ({slope_per_decade:.3f} °c/decade)"
)

# in many climate studies, the last ice age is often described as having a␣
↪global
# mean temperature of around 8°c, while the present global mean is around 15°c.
# this means the difference between an ice-age climate and the modern climate␣
↪is roughly 7°c at the global scale.
# korea is usually a bit cooler than the global average (for example, about 2°c␣
↪lower),
# so if we imagine that the global mean in 1850 was closer to 13°c, then korea␣
↪might
# have been around 11°c. to move from that kind of climate toward an␣
↪ice-age-like
# state, we can roughly think about a cooling of 3°c degrees.

temp_change_needed = 3.0  # °c

# take the absolute value of the slope, treating it as a possible rate of␣
↪cooling
annual_rate = abs(trend_coeffs[0])  # °c per year

# compute how many years would be required to change by 3°c at this linear rate
years_to_cool = temp_change_needed / annual_rate

print(f"Hypothetical years to cool by 3°C at this rate: {years_to_cool:.0f}␣
↪years")

# add labels, title, grid, and legend
ax.set_title(f"South Korea Temperature Anomaly Trend: {start_year} to␣
↪{end_year}")
ax.set_xlabel("Year")
ax.set_ylabel("Mean Temperature Anomaly (°C)")
ax.grid(True, linestyle=":", alpha=0.6)
ax.legend(loc="best")

plt.tight_layout()
plt.show()
```

```
estimated slope (1850-1960): -0.0281 °c per decade
Hypothetical years to cool by 3°C at this rate: 1069 years
```

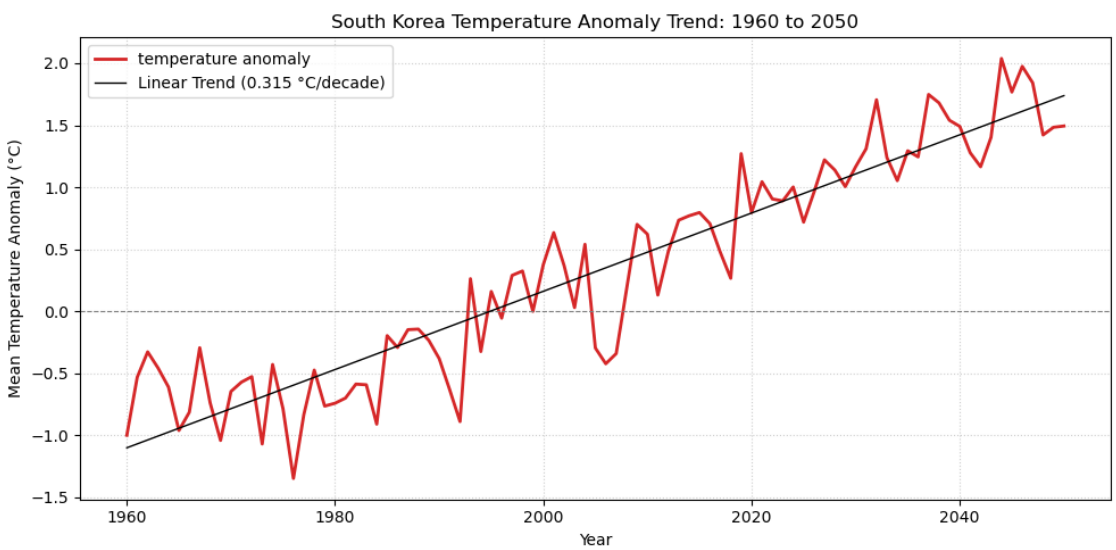


[31]:
```
# the industrial period in south korea roughly starts around the 1960s,
# when rapid economic growth and industrialization began (often called the rise␣
↪of the asian tigers)
start_year = 1960

# we look at the rest of the available data up to 2050
end_year = 2050

# select the years in the chosen range from the xarray dataset
industrial_selection = df_korea.sel(year=slice(start_year, end_year))

# convert the xarray object into a pandas dataframe for plotting and regression
industrial_selection_pd = industrial_selection.to_dataframe().reset_index()

# create a figure for the industrial period plot
fig, ax = plt.subplots(figsize=(10, 5))

# plot the temperature anomaly values year by year for the industrial period
ax.plot(
    industrial_selection_pd["year"],
    industrial_selection_pd["country_mean_temp"],
    color="tab:red",
    linewidth=2,
    label="temperature anomaly"
)

# add a horizontal line at 0°c anomaly as a baseline reference
ax.axhline(0, color="gray", linestyle="--", linewidth=0.8)
```

```
# prepare arrays for linear regression:
# X contains the years, and y contains the corresponding temperature anomalies
X = industrial_selection_pd["year"].values
y = industrial_selection_pd["country_mean_temp"].values

# fit a first-degree polynomial (a straight line) to estimate the modern trend:
# the regression finds coefficients for: anomaly = a * year + b
trend_coeffs = np.polyfit(X, y, 1)
trend_line = np.poly1d(trend_coeffs)

# convert the slope from °c per year to °c per decade, which is easier to␣
↪interpret
# positive slope means warming, negative slope would mean cooling
slope_per_decade = trend_coeffs[0] * 10

# print the estimated slope for this period
print(f"Estimated Slope ({start_year}-{end_year}): {slope_per_decade:.4f} °C␣
↪per decade")

# plot the fitted linear trend line on top of the data
ax.plot(
    X,
    trend_line(X),
    color="black",
    linestyle="-",
    linewidth=1,
    label=f"Linear Trend ({slope_per_decade:.3f} °C/decade)"
)

# the normal human body temperature is about 37°c. earlier, we imagined that
# korea in the mid-19th century might have been around 11°c on average, based on
# being a few degrees cooler than the global mean.
# if we compare 11°c (a cool climate) to 37°c (body temperature), the difference
# is 26°c. this is obviously not a realistic climate target, but it helps show
# how huge that temperature gap is compared to the small changes we usually talk
# about in climate science (like 1-3°c of warming)

temp_change_needed = 37 - 11  # °c difference between 11°c and human body␣
↪temperature

# take the absolute value of the estimated slope as a hypothetical warming rate
annual_rate = abs(trend_coeffs[0])  # °c per year

# compute how many years would be required to warm by this 26°c at the same␣
↪linear rate
years_to_human_body_temp = temp_change_needed / annual_rate
```

```
print(f"Hypothetical Years to reach 37°c at this rate:␣
↪{years_to_human_body_temp:.0f} years")

# add labels, title, grid, and legend for the plot
ax.set_title(f"South Korea Temperature Anomaly Trend: {start_year} to␣
↪{end_year}")
ax.set_xlabel("Year")
ax.set_ylabel("Mean Temperature Anomaly (°C)")
ax.grid(True, linestyle=":", alpha=0.6)
ax.legend(loc="best")

plt.tight_layout()
plt.show()
```

```
Estimated Slope (1960-2050): 0.3155 °C per decade
Hypothetical Years to reach 37°c at this rate: 824 years
```



1.6.1 Summary of Additional Analysis on South Korea's Temperature Data

Long-term warming trend:
South Korea warms at **+0.1102 °C per decade**, with rapid acceleration after 1950 in line with rising cumulative CO emissions.

Probability of extreme warm years:
Years exceeding a **+1.5 °C anomaly** occur **23%** of the time.

Temperature variability:
20-year rolling variability increases notably after the mid-20th century, with peaks around **1950–1970** and **2000–2020**, indicating rising climate instability.

Early historical trend (1850–1960):
A weak cooling signal (**–0.028 °C/decade**) dominated by natural variability.

Industrial-era trend (1960–2050):
A strong warming rate (**+0.3155 °C/decade**), more than 10× higher than the early period.

Lowest anomaly:
The lowest temperature anomaly appears in **1976** at **–1.348 °C**, marking a cold point before continued warming.

1.7 7. Make a command line interface to a prediction script

The inputs should include the name of a country (or global mean) and cumulative CO2 emissions. It should return the predicted temperature change relative to 1850. You can use the `argparse` package to do this. See here for documentation. Be sure to check for valid inputs.

Also provide the option to save the predictions to a CSV file.

This script should use the regression coefficients learned in the previous step so it doesn't have to use the full model output each time. You could store them in a numpy file, a pandas CSV file, or even JSON.

[32]:
```
# YOUR CODE HERE

import argparse
import pandas as pd
import sys
import json


def calculate_temp_change(
    temperature_change_per_gigaton_carbon,␣
 ↪cumulative_co2_emissions_gigatons_carbon
):
    """
    Predicts the change in temperature relative to the 1850 baseline.

    This function uses the core linear relationship:
    Temperature Change = Warming Rate * Cumulative CO2 Emissions
    """
    # calculate predicted temp change
    predicted_temp = (
        temperature_change_per_gigaton_carbon *␣
 ↪cumulative_co2_emissions_gigatons_carbon
    )

    return predicted_temp


def main():
```

```
# region --- cli argument parsing + definitions ---
# initialize arg parser
parser = argparse.ArgumentParser(
    description="Predict temperature change based on total carbon dioxide␣
↪emissions, using the regional warming rate (TCRE slope) if available, or the␣
↪global rate otherwise.",
    formatter_class=argparse.RawTextHelpFormatter,
)

# required argument
# the location the user wants a prediction for no spaces use "_" to␣
↪overwrite spaces and is case sensitive.
parser.add_argument(
    "location_code",
    type=str,
    help="The location code (e.g., 'South_Korea') or 'Global' for the␣
↪prediction.",
)

# required argument
# amount of pollution.
parser.add_argument(
    "cumulative_emissions_input_gigatons_carbon",
    type=float,
    help="Total Carbon Dioxide emissions (in Gigatons of Carbon) since 1850␣
↪for the prediction.",
)

# optional argument
# save file option
parser.add_argument(
    "-s",
    "--save_filename",
    type=str,
    help="The name of the CSV file to save the result to (e.g.,␣
↪'prediction_results.csv').",
)

# read args that user passed
user_args = parser.parse_args()
# endregion

# region --- load trained data ---
# coefficients file locaiton
coefficients_json_file = "trained_linear_coefficients.json"
```

```
    # intialize storage
    regression_coefficients = {}

    try:
        # load data into storage
        with open(coefficients_json_file, "r") as f:
            regression_coefficients = json.load(f)

        location_key = user_args.location_code

        # check if location key is in storage, if there is use it
        if location_key in regression_coefficients:
            print(f"Regional warming rate found for {location_key}.")
            warming_rate_slope = regression_coefficients[location_key]["slope"]
        # else there is no location key found, error out, no prediction can be␣
↪made
        else:
            raise KeyError(
                f"Error: The location provided: '{location_key}' was not found␣
↪in the 'trained_linear_coefficients.json' file"
            )
    # file doesnt exist
    except FileNotFoundError:
        print(
            f"Error: Coefficient file '{coefficients_json_file}' not found.␣
↪Please generate the file."
        )
        sys.exit(1)

    # property doesnt exist
    except KeyError as e:
        print(
            f"Error: The expected coefficient data (like 'slope') was not found␣
↪for the location. Key missing: {e}"
        )
        sys.exit(1)

    # not the expected coefficent file type
    except json.JSONDecodeError:
        print(f"Error: File '{coefficients_json_file}' is not a valid JSON␣
↪format.")
        sys.exit(1)

    # endregion

    # cant have negative emissions
    if user_args.cumulative_emissions_input_gigatons_carbon < 0:
```

```
        print(
            "Error: Total Carbon Dioxide emissions must be a non-negative␣
↪number (0 or greater)."
        )
        sys.exit(1)

    # --- Prediction Calculation ---
    # calculate emissions
    predicted_temp_change = calculate_temp_change(
        warming_rate_slope, user_args.cumulative_emissions_input_gigatons_carbon
    )

    # --- Output to Console ---
    print("\nPrediction Parameters:")
    print(f"Location: {user_args.location_code}")
    print(
        f"Total Carbon Dioxide Emissions: {user_args.
↪cumulative_emissions_input_gigatons_carbon:.2f} Gigatons of Carbon"
    )
    print("\nResults:")
    print(f"Predicted Temperature Change (since 1850): {predicted_temp_change:.
↪4f} °C")

    # --- save file ---
    if user_args.save_filename:
        try:
            # create a dataframe for easy csv export
            output_data = pd.DataFrame(
                [
                    {
                        "location": user_args.location_code,
                        "cumulative_co2_gigatons_carbon": user_args.
↪cumulative_emissions_input_gigatons_carbon,
                        "predicted_temperature_change_C": predicted_temp_change,
                    }
                ]
            )

            # destructive save file to specified location
            output_data.to_csv(user_args.save_filename, index=False,␣
↪header=True)

            print(f"Prediction successfully saved to: {user_args.
↪save_filename}")

        except Exception as e:
```

```
            print(f"Error saving to file: {e}")


if __name__ == "__main__":
    main()


# raise NotImplementedError()
```

```
usage: ipykernel_launcher.py [-h] [-s SAVE_FILENAME]
                             location_code
                             cumulative_emissions_input_gigatons_carbon
ipykernel_launcher.py: error: the following arguments are required:
cumulative_emissions_input_gigatons_carbon
```

```
An exception has occurred, use %tb to see the full traceback.

SystemExit: 2
```

```
/opt/conda/lib/python3.11/site-packages/IPython/core/interactiveshell.py:3585:
UserWarning: To exit: use 'exit', 'quit', or Ctrl-D.
  warn("To exit: use 'exit', 'quit', or Ctrl-D.", stacklevel=1)
```

[ ]:
```
!python predict_temp.py South_Korea 1500.0
```

1.7.1 No action is required below. Your answers for the project should be written above this line only.

*(Do **not** modify or delete the following "YOUR ANSWER HERE" block.)*

You may see a block below that says **"YOUR ANSWER HERE"**. This block is **not for you to fill in** — it is used internally by the TAs to grade the documentation, coding style, and code quality portions of your project.

- Documentation and Explanation — 10 points
- Code Documentation & Style — 5 points
- Code Organization & Structure — 5 points

YOUR ANSWER HERE

YOUR ANSWER HERE

YOUR ANSWER HERE